LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. Main Project
Student Name K. Anusha
Roll No. A24126552088
Date 29-10-2026

1. TITLE

Develop a Task Manager Web Application using Express, React and MongoDB (MERN Stack)

2. AIM

To design and develop a simple full stack Task Manager web application using Node.js with Express for the backend REST API, MongoDB for the database and React for the user interface, and to perform Create, Read, Update and Delete (CRUD) operations on tasks.

3. OBJECTIVE

The objectives of this project are:

- To understand the MERN stack and how its four parts work together.
- To create a REST API in Express with the routes GET, POST, PUT and DELETE.
- To connect Express to MongoDB using Mongoose and define a Task schema.
- To build a React user interface using functional components and the useState and useEffect hooks.
- To connect the React frontend to the Express backend using the fetch() API.
- To store tasks permanently in MongoDB so that they remain after a page refresh.
- To test each API route and each user interface operation.

4. THEORY

MERN Stack: MERN is a JavaScript stack made of MongoDB (database), Express (backend framework), React (frontend library) and Node.js (runtime). The same language, JavaScript, is used on both the client and the server.

MongoDB: A NoSQL document database that stores data as JSON-like documents (BSON) grouped into collections. MongoDB adds a unique _id field to every document automatically.

Mongoose: An Object Data Modelling (ODM) library for Node.js. A schema defines the fields, types and rules of a document, and a model gives ready-made methods such as find(), create(), findByIdAndUpdate() and findByIdAndDelete().

Express: A minimal Node.js web framework used to build the REST API. Each route handles an HTTP method and a URL. The express.json() middleware reads JSON request bodies and the cors middleware allows requests from other origins.

REST API and CRUD: CRUD operations are mapped to HTTP methods: POST creates a task, GET reads tasks, PUT updates a task and DELETE removes a task. Status codes such as 200, 201, 400 and 404 report the result.

React: A JavaScript library for building user interfaces from components. The useState hook stores data that changes, and the useEffect hook runs code when the component loads, which is used here to fetch the tasks.

Vite: A fast build tool and development server for React. Its proxy setting sends every request starting with /api to the Express server on port 5000, so the frontend can call the API without CORS problems.

5. ALGORITHM / PROCEDURE

- Install Node.js and MongoDB, and start the MongoDB server.
- Create the project folder with two sub-folders: server and client.
- In the server folder, run npm init and install express, mongoose, cors and dotenv.
- Store the port number and the MongoDB connection string in the .env file.
- Create server.js and connect to MongoDB using mongoose.connect().
- Define the Task schema with the fields title, completed and timestamps.
- Create the routes: GET /api/tasks, POST /api/tasks, PUT /api/tasks/:id and DELETE /api/tasks/:id.
- Start the server using npm run dev and check the message MongoDB connected.
- Create the React application in the client folder using Vite and set the proxy for /api in vite.config.js.
- Create the App component with states for the task list, the new title and the task being edited.
- Load all tasks from the API with useEffect when the page opens.
- Add the functions to add, update (complete or edit) and delete a task, and reload the list after each change.
- Style the application with App.css.
- Start the client using npm run dev and open http://localhost:5173 in the browser.
- Test every operation in the API and in the browser, and record the results.

6. PROGRAM

Project folder structure:

```
task-manager-mern/
├── server/
│   ├── package.json
│   ├── .env
│   └── server.js
└── client/
    ├── package.json
    ├── vite.config.js
    ├── index.html
    └── src/
        ├── main.jsx
        ├── App.jsx
        └── App.css
```

server/package.json

```
{
  "name": "server",
  "version": "1.0.0",
```

```
  "main": "server.js",
  "scripts": { "start": "node server.js", "dev": "node --watch server.js" },
  "dependencies": { "cors": "^2.8.5", "dotenv": "^16.4.5", "express": "^4.19.2",
"mongoose": "^8.5.0" }
}
```

server/.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/taskmanager
```

server/server.js

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

// ---- Model ----
const Task = mongoose.model(
  "Task",
  new mongoose.Schema(
    {
      title: { type: String, required: true, trim: true },
      completed: { type: Boolean, default: false },
    },
    { timestamps: true }
  )
);

// ---- Routes (CRUD) ----
app.get("/api/tasks", async (req, res) => {
  res.json(await Task.find().sort({ createdAt: -1 }));
});

app.post("/api/tasks", async (req, res) => {
  try {
    const task = await Task.create({ title: req.body.title });
    res.status(201).json(task);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

app.put("/api/tasks/:id", async (req, res) => {
  try {
    const { title, completed } = req.body;
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      { title, completed },
      { new: true, omitUndefined: true, runValidators: true }
    );
    if (!task) return res.status(404).json({ error: "Task not found" });
    res.json(task);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

```
app.delete("/api/tasks/:id", async (req, res) => {
  try {
    await Task.findByIdAndDelete(req.params.id);
    res.json({ message: "Deleted" });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ---- Start ----
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(process.env.PORT, () =>
      console.log(`Server running on http://localhost:${process.env.PORT}`)
    );
  })
  .catch((err) => console.error("DB connection error:", err.message));
```

client/package.json

```
{
  "name": "client",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build" },
  "dependencies": { "react": "^18.3.1", "react-dom": "^18.3.1" },
  "devDependencies": { "@vitejs/plugin-react": "^4.3.1", "vite": "^5.4.0" }
}
```

client/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: { proxy: { "/api": "http://localhost:5000" } },
});
```

client/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Task Manager</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

client/src/main.jsx

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./App.css";

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

client/src/App.jsx

```
import { useEffect, useState } from "react";

const API = "/api/tasks";

export default function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState("");
  const [editId, setEditId] = useState(null);
  const [editText, setEditText] = useState("");

  const load = async () => {
    const res = await fetch(API);
    setTasks(await res.json());
  };

  useEffect(() => {
    load();
  }, []);

  const addTask = async (e) => {
    e.preventDefault();
    if (!title.trim()) return;
    await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title }),
    });
    setTitle("");
    load();
  };

  const update = async (id, body) => {
    await fetch(`${API}/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    load();
  };

  const remove = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    load();
  };

  const saveEdit = async (id) => {
    if (editText.trim()) await update(id, { title: editText });
    setEditId(null);
  };

  const done = tasks.filter((t) => t.completed).length;

  return (
    <div className="container">
```

```
      <h1>Task Manager</h1>
      <p className="count">{done} of {tasks.length} completed</p>

      <form onSubmit={addTask} className="add">
        <input
          value={title}
          onChange={(e) => setTitle(e.target.value)}
          placeholder="Add a new task..."
        />
        <button type="submit">Add</button>
      </form>

      <ul>
        {tasks.map((t) => (
          <li key={t._id} className={t.completed ? "done" : ""}>
            <input
              type="checkbox"
              checked={t.completed}
              onChange={() => update(t._id, { completed: !t.completed })}
            />
            {editId === t._id ? (
              <input
                className="edit"
                value={editText}
                onChange={(e) => setEditText(e.target.value)}
                onKeyDown={(e) => e.key === "Enter" && saveEdit(t._id)}
                autoFocus
              />
            ) : (
              <span>{t.title}</span>
            )}
            <div className="actions">
              {editId === t._id ? (
                <button onClick={() => saveEdit(t._id)}>Save</button>
              ) : (
                <button onClick={() => { setEditId(t._id); setEditText(t.title);
}}>Edit</button>
              )}
              <button className="del" onClick={() => remove(t._id)}>Delete</button>
            </div>
          </li>
        ))}
      </ul>
      {tasks.length === 0 && <p className="empty">No tasks yet. Add one above!</p>}
    </div>
  );
}
```

client/src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f3f4f6; }
.container { max-width: 520px; margin: 40px auto; background: #fff; padding: 24px; border-
radius: 12px; box-shadow: 0 2px 10px rgba(0,0,0,.08); }
h1 { margin: 0; }
.count { color: #6b7280; margin: 4px 0 16px; }
.add { display: flex; gap: 8px; margin-bottom: 16px; }
.add input, .edit { flex: 1; padding: 10px; border: 1px solid #d1d5db; border-radius: 8px;
font-size: 15px; }
button { padding: 8px 12px; border: none; border-radius: 8px; background: #2563eb; color:
#fff; cursor: pointer; }
button.del { background: #dc2626; }
ul { list-style: none; padding: 0; margin: 0; }
```

```
li { display: flex; align-items: center; gap: 10px; padding: 10px 0; border-bottom: 1px
solid #e5e7eb; }
li span { flex: 1; }
li.done span { text-decoration: line-through; color: #9ca3af; }
.actions { display: flex; gap: 6px; }
.empty { text-align: center; color: #9ca3af; }
```

7. CODE EXPLANATION

Code	Explanation
`require("dotenv").config()`	Loads the PORT and MONGO_URI values from the .env file.
`app.use(cors());` `app.use(express.json())`	Enables cross-origin requests and lets Express read JSON request bodies.
`new mongoose.Schema({ title, completed }, { timestamps: true })`	Defines a task: title is a required trimmed string, completed is a Boolean (default false). createdAt and updatedAt are added automatically.
`app.get("/api/tasks")`	Reads all tasks from MongoDB, newest first (sort by createdAt descending).
`app.post("/api/tasks")`	Creates a task from req.body.title and returns it with status 201. A missing title gives status 400.
`app.put("/api/tasks/:id")`	Updates the title and/or completed field of the task with the given id. Returns 404 if the task is not found.
`app.delete("/api/tasks/:id")`	Deletes the task with the given id and returns a confirmation message.
`mongoose.connect(...).then(app.listen)`	Connects to MongoDB first and starts the server only after the connection succeeds.
`server: { proxy: { "/api": ... } }`	In vite.config.js, forwards the browser's /api requests to the Express server on port 5000.
`useState([...])`	Stores the task list, the new task title, the id of the task being edited and its text.
`useEffect(() => { load(); }, [])`	Fetches all tasks once when the page is first displayed.
`addTask(e)`	Stops the page reload, sends a POST request with the title, clears the input and reloads the list.
`update(id, body)`	Sends a PUT request. It is used for the checkbox (completed) and for saving an edited title.
`remove(id)`	Sends a DELETE request for the task and reloads the list.
`tasks.filter(t => t.completed).length`	Counts the completed tasks for the line “X of Y completed”.
`tasks.map(t => <li key={t._id}>...)`	Draws one list row per task with a checkbox, the title, and the Edit and Delete buttons. Completed tasks get the class done, which shows a strike-through.

8. TEST CASES AND EXECUTION DATA

Each operation is tested against the API (using Postman, Thunder Client or curl) and in the browser. The Actual Result and Status columns are filled in after the test is run.

Test Case	Operation	Input / Request	Expected Result	Actual Result	Status
TC-01	Start server	npm run dev (server folder)	MongoDB connected and server running on port 5000		
TC-02	Create	POST /api/tasks { "title": "Buy books" }	Status 201; task returned with _id and completed: false		
TC-03	Create (invalid)	POST /api/tasks { }	Status 400 with a validation error message		
TC-04	Read all	GET /api/tasks	Array of all tasks, newest first		
TC-05	Update status	PUT /api/tasks/:id { "completed": true }	Task returned with completed: true		
TC-06	Update title	PUT /api/tasks/:id { "title": "Buy notebooks" }	Task returned with the new title		
TC-07	Update (not found)	PUT /api/tasks/<unused id>	Status 404, “Task not found”		
TC-08	Delete	DELETE /api/tasks/:id	{ message: "Deleted" }; task no longer in the list		
TC-09	UI: add	Type a title and click Add	New task appears at the top; input box is cleared		
TC-10	UI: complete	Tick the checkbox of a task	Title is struck through; counter increases by 1		
TC-11	UI: edit	Click Edit, change the text, click Save	Task shows the updated title		
TC-12	UI: delete	Click Delete on a task	Task is removed from the list		
TC-13	Persistence	Refresh the browser page	All tasks are still displayed (stored in MongoDB)		

9. OUTPUT

The expected output of each test is given below. The _id values are generated automatically by MongoDB, so they are shown as '...' and will be different on every system.

Output 1 (TC-01): Starting the server

```
> npm run dev
MongoDB connected
Server running on http://localhost:5000
```

Output 2 (TC-02): Create a task

```
> POST /api/tasks   { "title": "Buy books" }
Status: 201 Created
{
  "_id": "...",
  "title": "Buy books",
  "completed": false,
  "createdAt": "2026-10-29T...",
  "updatedAt": "2026-10-29T...",
  "__v": 0
}
```

Output 3 (TC-03): Create with no title

```
> POST /api/tasks   { }
Status: 400 Bad Request
{ "error": "Task validation failed: title: Path `title` is required." }
```

Output 4 (TC-04): Read all tasks

```
> GET /api/tasks
[
  { "_id": "...", "title": "Buy books", "completed": false, ... }
]
```

Output 5 (TC-05): Mark as completed

```
> PUT /api/tasks/:id   { "completed": true }
{ "_id": "...", "title": "Buy books", "completed": true, ... }
```

Output 6 (TC-08): Delete a task

```
> DELETE /api/tasks/:id
{ "message": "Deleted" }
```

Output 7 (TC-09): Task Manager page with tasks added

[Paste screenshot here]

Output 8 (TC-10 and TC-11): Completed task and edit mode

[Paste screenshot here]

Output 9 (TC-04): MongoDB collection (tasks) shown in MongoDB Compass or mongosh

[Paste screenshot here]

10. RESULT

Thus, a simple Task Manager web application was developed using the MERN stack. The Express server provides a REST API for CRUD operations, Mongoose stores the tasks in MongoDB, and the React frontend lets the user add, view, edit, complete and delete tasks. The operations were tested through the API and the browser, and the tasks remained stored in the database after the page was refreshed.